Error Levels

Not all errors are created equal. Some are more serious than others. An error might be truly unrecoverable ("fatal") and will kill your program instantly, or it could be a warning… or even just informational in nature. Some of these errors occur during script execution and are *runtime*, some occur during the initial parsing of code and are *compile-time*, and PHP will even complain (or explode) during *startup* before it has even had a chance to look at your code.

Runtime errors are going to be your biggest headache; anything that kills your program during startup or compile time should not make it onto a production system (because you'll be checking for those before deployment, right?) and tend to be obvious in their cause. PHP runtime errors fall roughly into four categories: fatal errors, warnings, notices, and deprecation warnings (which aren't strictly speaking errors, just the language giving you prior notice that the next version of PHP is going to break whatever you are doing).

PHP has the following predefined constants for runtime errors…

Table 4-6. *Runtime error constants*

Constant	Definition
`E_ERROR`	Unrecoverable, terminates execution
`E_WARNING`	Recoverable, execution continues
`E_NOTICE`	Recoverable, informational: can occur during normal operation
`E_DEPRECATED`	Recoverable, code flagged for removal in future version
`E_USER_ERROR`	`E_ERROR` emitted by `trigger_error()`
`E_USER_WARNING`	`E_WARNING` emitted by `trigger_error()`
`E_USER_NOTICE`	`E_NOTICE` emitted by `trigger_error()`
`E_USER_DEPRECATED`	`E_DEPRECATED` emitted by `trigger_error()`
`E_RECOVERABLE_ERROR`	Legacy can be recovered with custom handler
`E_STRICT`	Recoverable, informational: suggestions for forward compatibility
`E_ALL`	Equivalent to the set of all error constants (runtime or otherwise)